

MedGuardAI: Safety-First Medical Assistant

Milestone #2 - Architecture & Implementation Details

1 Project Overview

Project Name: MedGuardAI

Team Name: MEDAI

Members: Andrei Moca, Bogdan Borodi, Calin Pauliuc, Alexandra Petrea, Ana Vaicum

GitHub Repository: <https://github.com/andreimoca/MedGuardAI>

1.1 Architecture Description

MedGuardAI is an AI-driven medical assistant powered by Agentic Retrieval-Augmented Generation (RAG). The system architecture comprises a FastAPI Python backend for RAG processing and Small Language Model (SLM) integration, paired with a React-based Single Page Application (SPA) frontend utilizing Vite and Framer Motion for an intuitive chat interface.

The core innovation lies in its **safety-first approach**: the system maintains strict guardrails by validating all queries against patient-specific context (age, weight, allergies, pre-existing conditions) and retrieving information exclusively from verified medical documentation stored in a local vector database built from official FDA drug labels.

2 Technical Architecture

2.1 Dataset Selection

Data Source: OpenFDA API (DailyMed Drug Labels)

API Endpoint: <https://api.fda.gov/drug/label.json>

2.1.1 Dataset Rationale

We selected the OpenFDA DailyMed dataset for several critical reasons:

1. **Official Authority:** FDA-approved drug labels represent the gold standard for medication information in the United States
2. **Structured Data:** The API provides well-structured JSON responses with clearly delineated sections
3. **Comprehensive Coverage:** Includes critical fields such as:
 - Boxed warnings (black box warnings for severe risks)
 - Contraindications (when NOT to use a drug)
 - Drug interactions
 - Dosage and administration guidelines
 - Adverse reactions

- Indications and usage
4. **Safety-Critical Information:** Each label undergoes rigorous FDA review, ensuring medical accuracy
 5. **Public Access:** Free API access with no authentication requirements, enabling reproducibility

2.1.2 Data Ingestion Pipeline

The ingestion process (`fetch_openfda.py`) queries the API with filters to prioritize drugs that have documented warnings, contraindications, or dosage information:

```

1 params = {
2     'search': '_exists_:boxed_warning OR _exists_:contraindications
3               OR _exists_:dosage_and_administration',
4     'limit': 50
5 }

```

Listing 1: OpenFDA API Query Strategy

Each drug label is processed and saved as an individual JSON file containing only the clinically relevant fields, filtering out marketing or administrative data. This ensures our vector database focuses exclusively on medical safety information.

2.2 Chunking Strategy

Method: RecursiveCharacterTextSplitter (LangChain)

Chunk Size: 1000 characters

Chunk Overlap: 200 characters

Separators: `["\n\n", "\n", ".", " ", ""]`

2.2.1 Design Rationale

1. Chunk Size (1000 characters):

- Large enough to capture complete medical statements (e.g., full contraindication descriptions)
- Small enough to ensure precise retrieval without excessive noise
- Optimized for sentence-transformer embedding model context windows

2. Chunk Overlap (200 characters):

- Prevents critical information from being split across chunk boundaries
- Ensures contextual continuity for multi-sentence warnings
- 20% overlap provides redundancy without excessive duplication

3. Hierarchical Separators:

- Prioritizes natural document boundaries (paragraph breaks)
- Falls back to sentence and word boundaries when needed
- Preserves semantic coherence of medical statements

This strategy is specifically optimized for **factual medical retrieval**, where precision and completeness are paramount. Smaller, focused chunks enable the retriever to pinpoint exact contraindications or dosage guidelines without retrieving irrelevant sections.

2.3 Vector Database Choice

Selected Database: ChromaDB

Persistence Mode: Local disk storage

Storage Path: backend/data/processed/chroma_db/

2.3.1 ChromaDB Selection Criteria

1. **Zero Configuration:** Embedded database requiring no separate server process
2. **Lightweight:** Minimal resource footprint suitable for local deployment
3. **LangChain Integration:** Native support for LangChain's retriever interface
4. **Persistence:** Saves embeddings to disk, eliminating need for re-embedding on restart
5. **Python-Native:** Pure Python implementation with excellent ecosystem compatibility
6. **Sufficient for MVP:** Handles 50-1000 document corpus efficiently without complex infrastructure

2.3.2 Retrieval Configuration

```
1 self.retriever = self.vectorstore.as_retriever(  
2     search_kwargs={"k": 3}  
3 )
```

Listing 2: Retriever Setup

We retrieve the **top 3 most semantically similar chunks** per query. This balance ensures:

- Sufficient context for the SLM to formulate comprehensive answers
- Minimal noise from irrelevant documents
- Reasonable token budget for the language model input

2.4 Model Choices

2.4.1 Embedding Model

Model: all-MiniLM-L6-v2 (Sentence Transformers)

Dimensions: 384

Context Length: 256 tokens

Selection Rationale:

- **Performance:** Fast inference on CPU, enabling quick document indexing and retrieval
- **Quality:** Proven performance on semantic textual similarity benchmarks
- **Size:** Lightweight (80MB) model suitable for local deployment
- **Domain Agnostic:** General-purpose embeddings work well for medical text without fine-tuning
- **Community Standard:** Widely adopted in production RAG systems

2.4.2 Small Language Model (SLM)

Model: Google Gemma-3-4b

Deployment: Local inference via LM Studio

API Endpoint: <http://127.0.0.1:1234/v1> (OpenAI-compatible)

Selection Rationale:

- **Local Execution:** No cloud dependencies, ensuring data privacy and offline capability
- **Efficient Size:** 4B parameters provide strong reasoning while maintaining reasonable inference speed
- **SLM Advantage:** Smaller model size enables fast inference on consumer hardware without sacrificing medical reasoning quality
- **Medical Reasoning:** Gemma models demonstrate strong performance on instruction-following and factual tasks
- **Temperature 0.0:** Deterministic generation reduces hallucination risk for safety-critical medical advice
- **Open Weights:** Freely available without API costs or rate limits

Alternative Considered: Initially prototyped with Qwen3.5-9b, but switched to Gemma-3-4b for improved inference speed on consumer hardware.

2.5 RAG Strategy

2.5.1 Pipeline Architecture

Our RAG implementation follows a **three-stage pipeline** with integrated safety mechanisms:

1. Pre-Retrieval Guardrail: Hard-coded emergency keyword detection

- Intercepts life-threatening queries before LLM invocation
- Keywords: anaphylaxis, overdose, cannot breathe, heart attack, chest pain
- Immediately returns emergency guidance without consulting the LLM

2. Semantic Retrieval:

- User query is embedded using all-MiniLM-L6-v2
- ChromaDB performs cosine similarity search
- Top 3 most relevant chunks retrieved from FDA documentation
- Patient context (age, weight, allergies, conditions) prepared for prompt injection

3. Contextualized Generation:

- System prompt emphasizes safety-first approach with strict instructions:
 - Never hallucinate dosages or medical facts
 - Base all answers on retrieved documentation
 - Default to "I don't know" when information is insufficient
 - Integrate patient-specific risk factors into reasoning
- Retrieved documents + patient profile + user query combined in structured prompt
- Gemma-3-4b SLM generates grounded response with medical reasoning

2.5.2 Prompt Engineering

The system prompt is meticulously crafted to enforce medical safety constraints:

```
1 """You are MedGuardAI, a highly specialized clinical assistant.
2
3 CRITICAL INSTRUCTIONS:
4 1. Never Hallucinate Dosages: Base answers entirely on
5   retrieved documentation
6 2. "I Don't Know" Fallback: State insufficient information
7   when context is inadequate
8 3. Emergency Trigger: Identify life-threatening symptoms
9 4. Context Constraints: Integrate patient demographics
10 (Age, Weight, Allergies)"""
```

Listing 3: System Prompt Structure (Excerpt)

2.5.3 RAG Strategies Considered

Strategy	Advantages	Status
Basic RAG	Simple, reliable, low latency	Implemented (v1.0)
Query Rewriting	Better retrieval for ambiguous queries	Planned
Multi-Query RAG	Increased recall through query variations	Planned
Hybrid Search	Combines semantic + key-word search	Planned
Agentic RAG	SLM decides retrieval strategy dynamically	Future work
Re-ranking	Second-stage relevance scoring	Future work

Table 1: RAG Strategy Roadmap

3 Implementation Details

3.1 Backend Stack

- **Framework:** FastAPI 0.100+ (async-first, OpenAPI documentation)
- **RAG Orchestration:** LangChain 0.1+
- **Embeddings:** HuggingFace Transformers + Sentence Transformers
- **Vector Store:** ChromaDB 1.7+
- **SLM Inference:** LM Studio (local server) with OpenAI-compatible API
- **Testing:** Pytest for unit and integration tests

3.2 Frontend Stack

- **Framework:** React 18 with Vite build tooling
- **UI/UX:** Framer Motion for fluid animations
- **API Communication:** Axios/Fetch for RESTful backend integration

3.3 API Design

Primary Endpoint: POST /api/v1/ask

Request Schema:

```
1 {  
2   "query": "Is it safe to take ibuprofen?",  
3   "patient_context": {  
4     "age": 45,  
5     "weight": 80,  
6     "allergies": ["Penicillin"],  
7     "conditions": ["Hypertension"]  
8   }  
9 }
```

Listing 4: API Request Format

Response Schema:

```
1 {  
2   "answer": "Based on the retrieved documentation...",  
3   "status": "success" // or "emergency"  
4 }
```

Listing 5: API Response Format

4 Safety Mechanisms

4.1 Multi-Layer Guardrails

1. **Pre-SLM Emergency Detection:** Keyword-based filter intercepts critical queries
2. **Grounded Generation:** System prompt enforces strict adherence to retrieved context
3. **Mandatory Patient Context:** All queries require age, weight, allergies, and conditions
4. **Hallucination Prevention:** Temperature set to 0.0 for deterministic, factual responses
5. **Explicit Uncertainty Handling:** SLM instructed to admit when information is insufficient

4.2 Testing Strategy

Comprehensive test suite (`test_agent_safety.py`) validates:

- Emergency guardrail triggers correctly on life-threatening symptoms
- Safe queries bypass guardrails appropriately
- Patient context formatting prevents prompt injection
- API endpoints return correct status codes

5 System Workflow

1. **User Input:** Patient submits query via React chat interface with demographic context
2. **API Reception:** FastAPI validates request schema using Pydantic models
3. **Emergency Check:** Pre-SLM guardrail scans for critical keywords
4. **Semantic Search:** Query embedded and matched against ChromaDB vector store

5. **Context Retrieval:** Top 3 relevant drug label chunks retrieved
6. **Prompt Construction:** System prompt + patient profile + retrieved docs + query combined
7. **SLM Inference:** Gemma-3-4b generates grounded response via LM Studio
8. **Response Delivery:** Answer returned to frontend with safety status flag

6 Performance Considerations

6.1 Embedding Generation

- **Model Size:** 80MB (all-MiniLM-L6-v2)
- **Inference:** CPU-optimized, <100ms per query embedding
- **Indexing:** One-time cost during vector DB build

6.2 SLM Inference

- **Model Size:** 4GB (Gemma-3-4b quantized)
- **Latency:** 1-3 seconds per query on consumer hardware (M1/M2 Mac, RTX 3060+)
- **Privacy:** Fully offline inference, no data leaves local machine

7 Future Enhancements

7.1 RAG Improvements

- **Query Decomposition:** Break complex multi-part questions into sub-queries
- **Self-Querying:** Enable SLM to generate structured metadata filters
- **Hybrid Search:** Combine semantic embeddings with BM25 keyword search
- **Citation Generation:** Return source drug label references with answers

7.2 Safety Enhancements

- **NeMo Guardrails:** Replace keyword-based emergency detection with trained classifier
- **Output Validation:** Post-generation fact-checking against retrieved context
- **Uncertainty Quantification:** Confidence scores for model predictions
- **Human-in-the-Loop:** Flagging mechanism for ambiguous cases requiring physician review

7.3 Scalability

- **Database Migration:** Evaluate Qdrant or Weaviate for larger corpora (>10K documents)
- **Batch Processing:** Parallel document ingestion for faster indexing
- **Caching:** Redis integration for frequently asked questions

8 Evaluation Metrics (Planned)

- **Retrieval Accuracy:** Precision@3, Recall@3 on hand-labeled test queries
- **Answer Quality:** ROUGE-L, BERTScore against reference answers
- **Safety Compliance:** Binary classification on hallucination detection
- **Latency:** P50, P95, P99 end-to-end response times

9 Conclusion

MedGuardAI implements a production-grade RAG architecture with an uncompromising focus on medical safety. By grounding all responses in FDA-verified documentation and enforcing strict guardrails, the system minimizes hallucination risk while maintaining conversational fluency. The modular design enables iterative improvements to retrieval strategies and safety mechanisms without architectural refactoring.

The GitHub repository contains complete implementation details, setup scripts for reproducible deployment, and comprehensive test coverage to validate safety-critical functionality.